



BookMate Admin

Web Development Work Sample

Prepared by Dilpreet Choudhary

Selected project sample for the Web Site Manager / Developer Student application

Project overview

BookMate Admin is an admin-facing interface for reviewing uploaded books, managing users, and monitoring order activity. The submitted project uses component-based page structure, routing, interactive state, filtering, status workflows, and custom CSS styling.

React

TypeScript

CSS

React Router

Vite

What this sample demonstrates

- Structured multi-page web interfaces with clear navigation and content organization.
- Interactive controls for reviewing, filtering, approving, rejecting, and managing records.
- Reusable frontend components, state-driven behavior, and route-based navigation.
- CSS-based visual design with tables, status badges, modals, forms, and responsive rules.

This PDF is a selected sample from the larger BookMate project files provided for the application.

1. Dashboard & Content Review

Interface preview reconstructed from the submitted BookMate source files

Admin

Dashboard

User

Orders

Logout

Uploaded Books

Sort by Price

All Categories

All Status

Details	Price	Status	Comments	Actions
CSTEXT Introduction to Algorithms Author: Thomas H. Cormen Publisher: MIT Press Genre: Computer Science	₹999	Pending	Core CS textbook	Approve Reject
ECON Principles of Economics Author: N. Gregory Mankiw Publisher: Cengage Genre: Economics	₹850	Pending	1st-year syllabus	Approve Reject
CHEM Organic Chemistry Author: Paula Y. Bruice Publisher: Pearson Genre: Chemistry	₹1,250	Pending	UG science	Approve Reject

Interface preview based on submitted BookMate source files

Key functionality in the submitted code

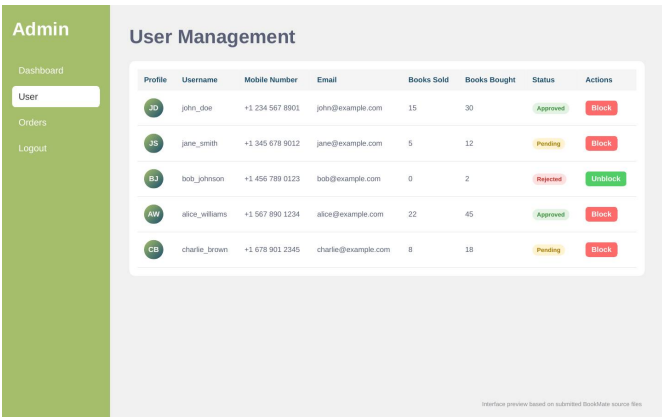
- Filters uploaded books by genre and review status, with price sorting from low-to-high or high-to-low.
- Uses approval and rejection actions with status changes tracked in component state.
- Includes a book-details modal, image navigation, and a rejection-reason workflow with an optional revised price.
- Organizes book metadata such as author, publisher, genre, price, status, and comments into a consistent review layout.

Role relevance

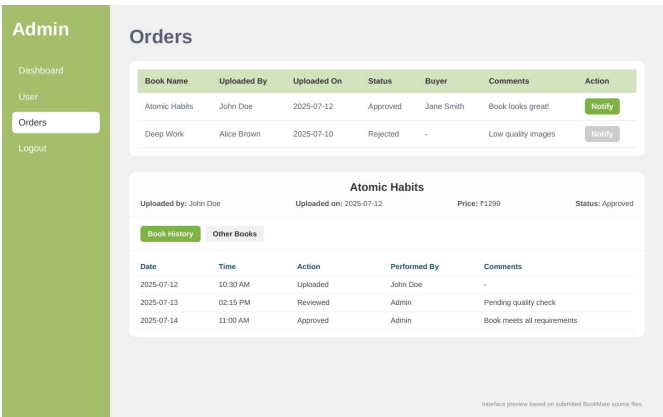
Shows page maintenance, structured content presentation, usability-focused controls, and troubleshooting-oriented review workflows - skills that transfer directly to maintaining and improving a content-driven website.

2. User & Order Management

Additional interface views from the submitted project



User management view



Orders and history view

User management

- Displays profile, contact, transaction counts, approval status, and block/unblock controls in a structured table.
- Opens user details in a tabbed popup with ordered books, uploaded books, and address history.
- Uses state updates to reflect user-management actions without reloading the page.

Order management

- Tracks book name, uploader, upload date, current status, buyer, comments, and context-specific actions.
- Provides modal details with book history and other-book tabs, supporting review of related information in one place.
- Uses status-aware button behavior and modal navigation to keep administrative workflows clear and organized.

Frontend skills demonstrated

Information architecture, navigation, interactive tables, state management, conditional UI behavior, forms, status indicators, modal interfaces, and custom CSS.

3. Selected Code & Technical Approach

Short excerpts from the files provided with this sample

Login flow: state, validation, async service call, and navigation

```
const [username, setUsername] = useState("");
const [password, setPassword] = useState("");

const handleLogin = async () => {
  if (username && password) {
    const result = await loginUser(username, password);
    if (result.token) {
      navigate("/Dashboard");
    }
  }
};
```

Route-based page structure

```
<Routes>
  <Route path="/" element={<LoginPage />} />
  <Route path="/dashboard" element={<Dashboard />} />
  <Route path="/orders" element={<Orders />} />
  <Route path="/user" element={<User />} />
</Routes>
```

Technical takeaways

- Component-based organization separates login, dashboard, user, and order responsibilities.
- React state controls filters, status changes, modal visibility, form values, and user actions.
- Custom CSS defines layout, visual hierarchy, buttons, tables, badges, overlays, and responsive behavior.
- The interface is organized around administrative tasks and clear content review flows.

Application connection

This sample highlights frontend development, content organization, usability, and interface maintenance. These are directly relevant to updating pages, improving user experience, testing site behavior, and troubleshooting web interfaces.